

WEB 1 – IDOR (Insecure Direct Object Reference)

Deskripsi

Website memiliki URL: `http://ctf.local/profile?id=102`

User login sebagai ID 102.

Jika diganti menjadi: `?id=101`

Muncul:

Welcome admin

Flag: `flag{hidden_admin}`

Catatan:

Konsep ini termasuk kategori **Broken Access Control** menurut OWASP.

Apa Itu IDOR?

IDOR (Insecure Direct Object Reference) terjadi ketika:

Aplikasi menggunakan parameter langsung (seperti ID) untuk mengakses data TANPA memverifikasi apakah user berhak mengakses data tersebut.

Artinya:

User bisa mengakses data milik orang lain hanya dengan mengganti angka ID.

Studi Kasus Challenge

URL normal : `http://ctf.local/profile?id=102`

User login sebagai : ID = 102

Ketika diubah menjadi : `http://ctf.local/profile?id=101`

Server menampilkan :

Welcome admin

Flag: `flag{hidden_admin}`

Mengapa Bisa Terjadi?

Kemungkinan besar source code backend seperti ini:

```
@app.route("/profile")
def profile():
    user_id = request.args.get("id")
```

```
user = db.execute("SELECT * FROM users WHERE id = ?", (user_id,))  
return render_template("profile.html", user=user)
```

Masalahnya:

Server tidak mengecek apakah user yang sedang login adalah pemilik ID tersebut.
Dia hanya mengambil ID dari URL dan langsung menampilkan data.

Kenapa Ini Bahaya?

Karena attacker bisa:

- Mengakses akun admin
- Mengakses data user lain
- Melihat email, password hash, dll
- Mengambil flag dalam konteks CTF

Langkah Eksploitasi (Dalam CTF)

1. Login sebagai user biasa

Misalnya ID = 102

2. Perhatikan URL

/profile?id=102

3. Coba ubah angka ID

?id=101

?id=100

?id=1

4. Server tetap menampilkan data

Berarti tidak ada validasi akses.

Ketika ID = 101:

Welcome admin

Flag: flag{hidden_admin}

Kenapa Disebut "Direct Object Reference"?

Karena:

- "Object" = data user di database

- "Direct Reference" = langsung lewat parameter id
- Tanpa kontrol akses

Cara Mencegah IDOR

- **Cara Salah (Rentan)**

```
user_id = request.args.get("id")
```

Server percaya penuh pada input user.

- **Cara Benar**

Gunakan ID dari session/login, bukan dari URL.

Contoh aman:

```
@app.route("/profile")
def profile():
    user_id = session["user_id"] # Ambil dari session login
    user = db.execute("SELECT * FROM users WHERE id = ?", (user_id,))
    return render_template("profile.html", user=user)
```

Atau lakukan validasi:

```
if request.args.get("id") != session["user_id"]:
    return "Unauthorized", 403
```

Versi Dunia Nyata

Kasus seperti ini pernah terjadi di banyak aplikasi besar, termasuk:

- Sistem e-learning
- Portal kampus
- Aplikasi bank
- Marketplace

Karena developer hanya fokus fungsi, bukan kontrol akses.

Ringkasan Alur Eksploitasi

Langkah	Penjelasan
1	Login sebagai user biasa

2	Perhatikan parameter ID
3	Ubah ID secara manual
4	Server tetap menampilkan data
5	Berhasil → IDOR

